# COMP4135 SEM Weekly Report 2

## Group 2 (Tactical Cards)

### Team members

Guangjun HU
Hongshuo HU
Cheng YE
Zehan CHEN

School of Computer Science
University of Nottingham

**Submission Date: 03/04/2026**

# Contents


**1 Specification and Design Decisions** **2**
- 1.1 Requirements Validation . . . 2
- 1.2 Architectural/System Diagram . . . 3

**2 Prototyping Process** **5**
- 2.1 Prototype . . . 5
- 2.2 Design Validation . . . 8

**3 Implementation Strategy** **9**
- 3.1 Code Structure . . . 9
- 3.2 Critical Implementation Choices . . . 10

**4 Summary and Reflections** **12**
- 4.1 Team Members and the Division of Labor . . . 12
- 4.2 Completed Process . . . 12
- 4.3 Future Plan . . . 13

**Reference** **14**

**A Meeting Minutes and Working Records** **15**
- A.1 Recovered Checkpoint (15/03/2026) . . . 15
- A.2 WR2 Evidence Freeze Note . . . 16
- A.3 Supporting Record Index . . . 16

# Chapter 1

# Specification and Design Decisions

This report focuses on how the Tactical Cards concept moved from the WR1 proposal into a constrained, demonstrable Unity prototype. The emphasis of WR2 is not to restate the full WR1 requirements, but to validate which requirements survived contact with implementation, which ones were simplified, and which ones were deliberately deferred to later stages. The report is intentionally anchored to one frozen prototype snapshot, recorded in the project record `wr2-snapshot-freeze-2026-04-02.md`. This means the chapter describes the current demo as it exists in the frozen WR2 evidence set, and separates target-state ambitions from implemented behaviour.

## 1.1 Requirements Validation

The current prototype validates the core loop of selecting a unit, drawing cards, playing a limited number of command cards, ending the turn, and observing an enemy response. However, several presentation and completion-oriented requirements remain partial rather than finished. The key WR1 to WR2 validation outcomes are:

- **6x6 tactical battle board.** Status: retained and implemented. Evidence: the current Unity scene and prototype screenshots. Reason: a single 6x6 scene is large enough to demonstrate positioning without expanding into campaign-scale scope.
- **Hand zone, turn controls, and combat information in one interface.** Status: retained and implemented. Evidence: the current `BattleUI` panel and screenshots. Reason: a left-side control panel plus in-world board keeps the prototype readable for reporting and demonstration.
- **Move, Strike, Guard, and Push as the starter card set.** Status: retained and narrowed. Evidence: the fallback deck and explicit card resolution logic. Reason: these four cards cover movement, damage, defense, and displacement with minimal rules overhead.

- **Card draw, discard, and fixed plays per turn.** Status: retained and implemented. Evidence: current turn flow and deck management logic. Reason: three cards drawn per turn and two playable actions proved easier to demonstrate than a wider energy or deckbuilding model.

- **Enemy units perform visible actions during the enemy phase.** Status: retained but simplified. Evidence: deterministic enemy movement and attack summary output. Reason: the prototype validates enemy response without requiring a broader AI system.

- **Clear visual and animation feedback for all actions.** Status: modified and partial. Evidence: tile highlights, card tinting, selection state, and status log. Reason: bespoke combat animation was reduced to lightweight prototype cues to keep WR2 scope under control.

- **Win/lose handling ends the combat session cleanly.** Status: retained and implemented in minimal form. Evidence: the current turn manager now stops the battle loop when one side is eliminated, and the battle UI surfaces a textual victory/defeat outcome. Reason: the team fixed the earlier accuracy gap with a narrow runtime change, while leaving richer result presentation for later polish.

- **Minimal onboarding cues for first interactions.** Status: partial. Evidence: the current UI gives a status prompt such as "Select a player unit," but not a fuller guided onboarding flow. This gap was confirmed during WR2 validation.

- **Structured repository workflow and traceable progress records.** Status: partially implemented and now evidenced through the cleaned GitHub workflow. Evidence: merged pull requests on the canonical repository, the collaboration-history note, and the process evidence package under `doc/src/project-records/process/`. Reason: the repository workflow is now visible in GitHub history, but task-board captures and stronger attendance evidence are still weaker than the repository trail itself.

The validated WR2 scope is therefore narrower than the original concept sketched in WR1 and the one-pager [3]. The frozen prototype snapshot commits to a single battle scene, two player units, one enemy response loop, four starter cards, clear board-state feedback, and truthful battle-end termination with text-based outcome reporting. Features such as a larger card pool, multi-scene structure, deckbuilding, progression, and a more expressive tutorial flow remain outside the current implementation window [1]. They are discussed as deferred work rather than being blended into the current-state description.

## 1.2 Architectural/System Diagram

The current system architecture follows the "single battle, shallow object model" direction set out in the architecture constraints and first-pass script notes [1, 2].

Figure 1.1 reflects the current code modules rather than the more speculative abstraction layer implied by some earlier planning material.

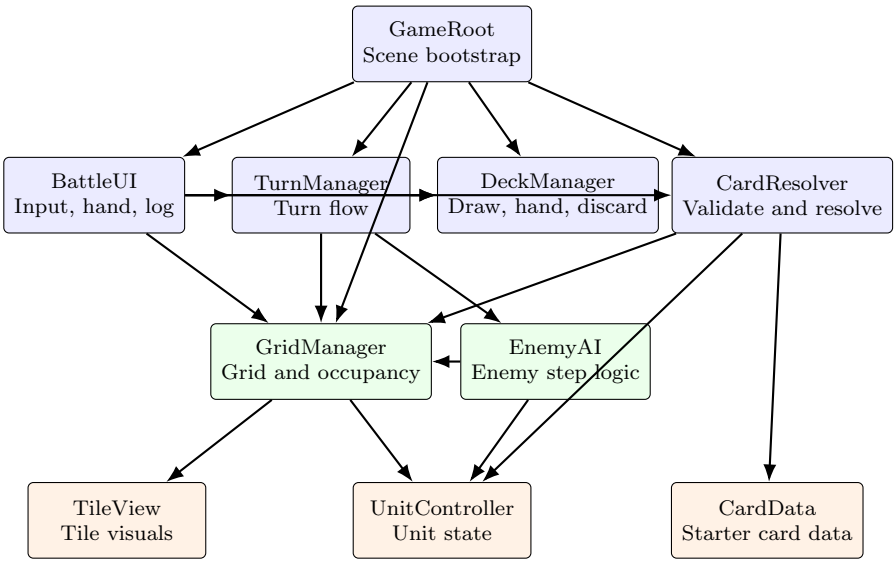


Figure 1.1: Current WR2 runtime architecture for the Tactical Cards prototype

The control flow begins at `GameRoot`, which wires together the scene managers and registers the units already placed in the scene. `TurnManager` owns turn state, remaining card plays, and the switch between player and enemy phases. `BattleUI` is intentionally thin: it handles card selection, world clicks, and status output, but it delegates gameplay decisions to the deck, grid, and card systems.

The data flow is intentionally conservative. `DeckManager` owns draw, hand, and discard state. `CardResolver` is the only module allowed to interpret `CardType`, which keeps card behaviour explicit and easy to trace. `GridManager` owns tile generation, occupancy, and reachability, while `UnitController` stores runtime unit state such as hit points, team, guarding, and grid position. This arrangement matches the current prototype goal: one readable vertical slice rather than a general-purpose tactics engine.

# Chapter 2

# Prototyping Process

The prototype did not begin as a polished scene. It moved through three practical layers: an early game concept in the one-pager [3], a narrowed architectural constraint document that removed non-essential systems [1], and a first-pass script list that defined the minimum compilable gameplay skeleton [2]. WR2 records the point where these design decisions became a playable vertical slice. The narrative in this chapter follows the frozen WR2 snapshot rather than the team's longer-term target state, so partial and deferred items are kept explicit.

## 2.1 Prototype

Figure 2.1 shows a retrospective low-fidelity wireframe rebuilt from the WR1 scope and the current scene layout. It is explicitly marked as retrospective because no separate low-fidelity mockup was archived in the local workspace before prototype implementation began. The wireframe is still useful in WR2 because it clarifies the intended information zones: turn state, hand controls, status feedback, and the board itself.

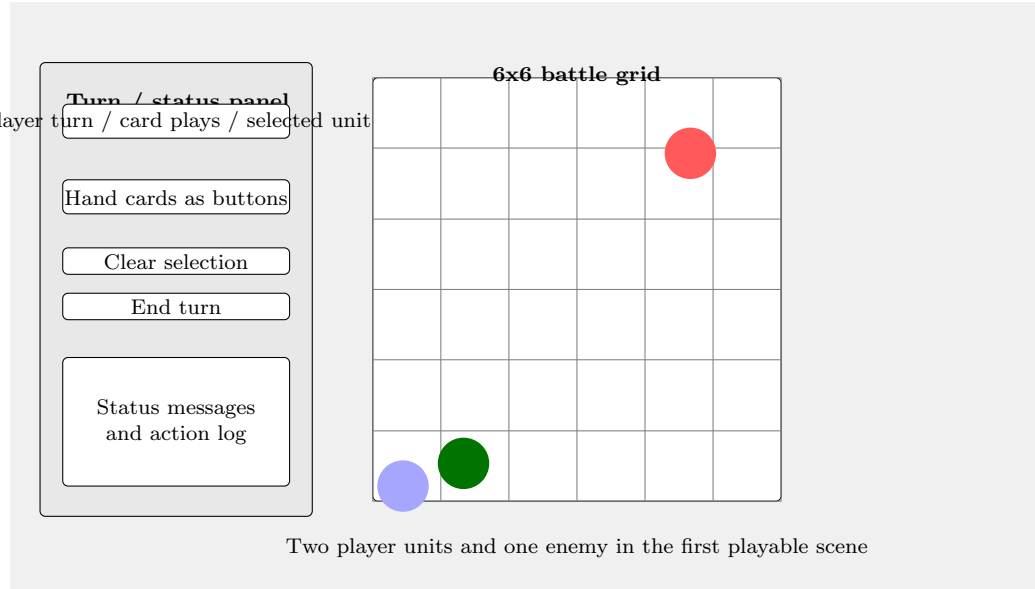


Figure 2.1: Retrospective low-fidelity wireframe reconstructed from WR1 scope and the current prototype layout

The high-fidelity prototype evidence comes from the working Unity scene. Figure 2.2 shows the current battle board and left-side battle UI, while Figure 2.3 isolates the in-world battle scene used as the basis for interaction and screenshot capture.

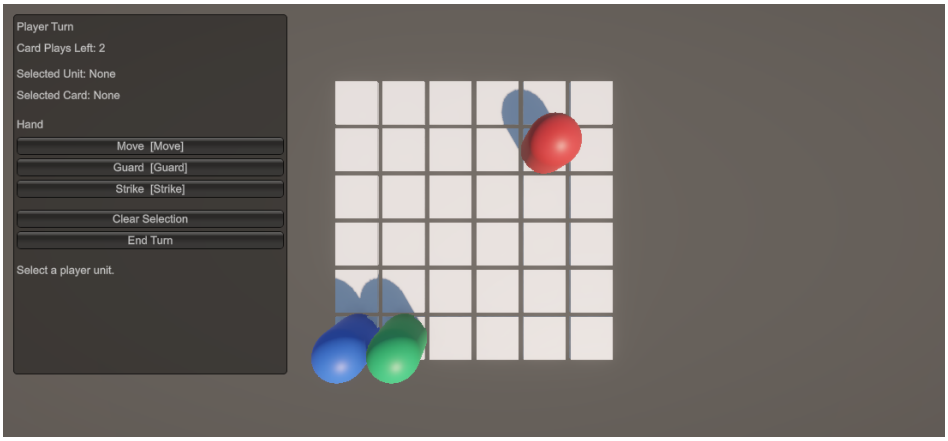


Figure 2.2: Current high-fidelity prototype with battle UI, starter hand, and 6x6 board

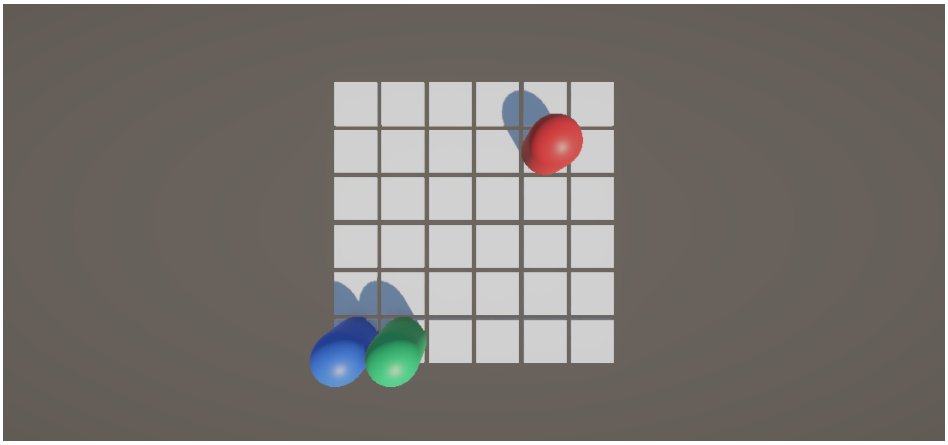

Figure 2.3: Board-only view of the current playable battle scene

The current frozen prototype feature state can be summarised as follows:

- **Implemented.** 6x6 battle grid, unit selection, starter hand and card draw, Move/Strike/Guard/Push resolution, enemy response in simplified form, board highlight feedback, and battle-end gating with textual victory/defeat reporting.
- **Partial.** Onboarding cues and feedback emphasis. The UI prompts the player, but it does not yet teach the full interaction sequence or explain all board highlight meanings. The battle-end result is currently truthful and visible, but still lightweight rather than polished.
- **Initial automated verification now exists.** A committed EditMode test suite checks three battle-end scenarios in the turn manager, which gives WR2 at least a minimal automated validation layer instead of a purely informal claim set.
- **Deferred.** Broader automated coverage, stronger external playtesting, and richer audiovisual result presentation remain for the next reporting cycle.

Several implementation constraints shaped the prototype:

- The UI is a hybrid prototype solution rather than a finished production interface. It prioritises immediate readability and low implementation cost over long-term extensibility.
- Input is routed through the current Input System plus physics raycasts, which made it possible to connect button-driven card choice to in-world tile and unit selection quickly.
- Enemy behaviour is intentionally deterministic. This made debugging, validation, and reporting easier than a more expressive AI would have.
- The prototype proves the battle loop and feedback model first. Follow-up work now extends the quality of battle-end presentation, onboarding flow, and automated coverage rather than inventing new core systems.

## 2.2 Design Validation

WR2 design validation was carried out as an internal heuristic walkthrough of the frozen prototype snapshot on 01/04/2026. Because the local workspace does not preserve a separate user-test archive yet, this stage focused on structured scenario review rather than formal external playtesting. The walkthrough checked three scenarios aligned with the current demo goals: selecting a player unit, playing a card against a valid target, and ending the turn to observe enemy behaviour. The record is stored as `validation/wr2-heuristic-walkthrough-2026-04-01.md`, and it remains labelled as a heuristic walkthrough rather than being upgraded into a stronger playtest claim. After that walkthrough, one narrow runtime fix and an EditMode test pass were added to correct the battle-end accuracy gap without widening the prototype scope.

The key validation findings are:

1. **Scenario: first interaction and unit selection.** Finding: the player can infer the next step from the left-side status prompt, but there is no stronger onboarding cue on the board itself. Severity: medium. Proposed action: add a clearer first-turn prompt or highlight the expected first click. Status: planned.

2. **Scenario: play a card and confirm its target.** Finding: card meanings are readable, but the board does not explain move versus attack highlight colours directly. Severity: medium. Proposed action: add a tiny legend or one-line prompt when a card is selected. Status: planned.

3. **Scenario: end turn and observe enemy response.** Finding: enemy actions were readable through summary text, but the battle loop initially lacked a clear end-state gate and outcome cue. Severity: high. Proposed action: add explicit battle-end termination and visible victory/defeat feedback before final WR2 closeout. Status: applied in narrow form through the prototype-accuracy fix; stronger post-action emphasis remains planned.

These findings directly influenced the wording of WR2. The report does not claim a finished tutorial flow, a polished result screen, or full audiovisual combat feedback. Instead, it presents the prototype as a validated vertical slice with a working end-state gate, textual outcome feedback, and an initial automated check around the corrected battle-end behaviour. This keeps the report aligned with actual evidence rather than idealised design intent.

# Chapter 3

# Implementation Strategy

The implementation strategy is based on one principle: keep the prototype readable enough that every major gameplay responsibility can be traced to a small number of Unity scripts. This section therefore focuses on code structure and the concrete trade-offs that shaped the current prototype.

## 3.1 Code Structure

The current gameplay scripts are organised into a small set of folders under `coursework/Assets/Scripts`. The directory tree below reflects the current runtime structure used by the prototype.

```
Assets/Scripts
|-- Cards
|   |-- CardData.cs
|   |-- CardResolver.cs
|   `-- DeckManager.cs
|-- Common
|   `-- GameEnums.cs
|-- Core
|   |-- GameRoot.cs
|   `-- TurnManager.cs
|-- Grid
|   |-- GridManager.cs
|   `-- TileView.cs
|-- UI
|   |-- BattleUI.cs
|   `-- CardButtonView.cs
`-- Units
    |-- EnemyAI.cs
    `-- UnitController.cs
```

This structure intentionally mirrors responsibilities rather than future-proof architecture. The `Core` folder owns scene bootstrapping and turn flow. `Grid` owns tile generation, occupancy, and range queries. `Cards` owns card data, deck state, and explicit card resolution. `Units` owns health, team, and enemy decision logic. `UI` acts as the glue between player input and runtime state.

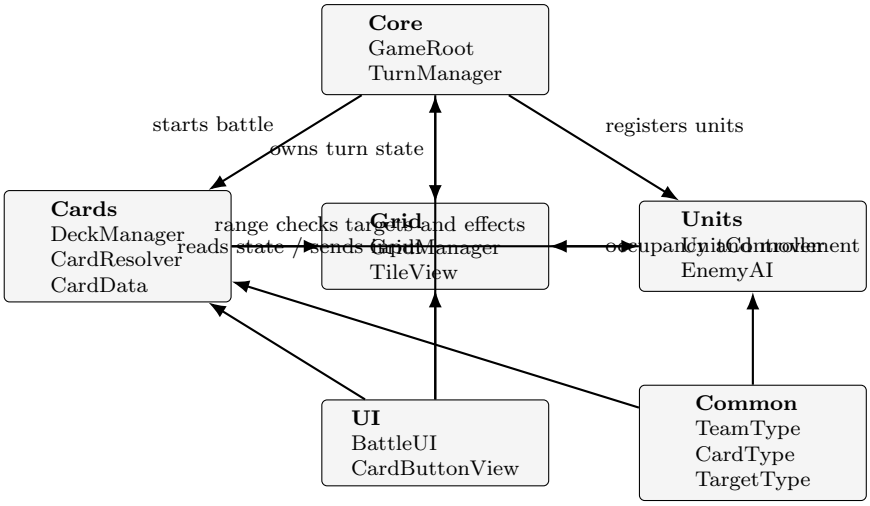


Figure 3.1: Simplified module dependency and responsibility map

Figure 3.1 shows that dependencies flow inward toward a small number of managers instead of outward into a large inheritance tree. This is consistent with the project goal of a reportable, traceable prototype rather than a content-heavy tactics framework.

## 3.2 Critical Implementation Choices

The most important implementation choices in the current prototype are:

- **Single self-contained battle.** Options considered: one battle scene versus multi-scene campaign flow. Chosen option: one battle scene, because it provides a complete vertical slice for WR2 without spending time on navigation and persistence. Trade-off: broader progression is postponed.

- **Shared unit runtime model.** Options considered: one shared controller versus separate player/enemy hierarchies. Chosen option: one shared `UnitController` so that health, guarding, movement, and grid state remain easy to trace. Trade-off: the model is less specialised than a longer-term production structure.

- **Explicit card resolver.** Options considered: `enum`-driven explicit logic versus a generic card-effect hierarchy. Chosen option: one resolver that interprets `CardType`, because the starter set is small and benefits from readable switch-based logic. Trade-off: a larger future card set would eventually need refactoring.

- **Deterministic enemy AI.** Options considered: nearest-target movement versus broader tactical AI. Chosen option: deterministic nearest-target behaviour, because it is predictable, testable, and easy to explain in WR2. Trade-off: the enemy behaviour remains strategically shallow.

- **Prototype-first UI and input path.** Options considered: a full Unity UI pipeline versus the current hybrid prototype UI with Input System and

board raycasts. Chosen option: the hybrid path, because it was faster to wire into the existing scene and sufficient for WR2 reporting. Trade-off: UI consistency and polish remain limited.

- **GitHub-first collaboration after repository cleanup.** Options considered: continue closing WR2 on a mixed local clone versus migrate report-closeout work onto a cleaned GitHub workflow with explicit PR rules. Chosen option: move current collaboration onto the cleaned GitHub repository, record the review policy, and keep earlier local/Overleaf/WeChat-era activity in supporting notes. Trade-off: current closeout work is now traceable through merged pull requests, but some earlier process evidence still relies on reconstructed notes and the task-board trail remains weaker than the repository history.

The most important strategic choice was to treat the prototype as implementation evidence rather than as the start of a fully general tactics engine. That choice explains why the runtime modules are explicit, why the card system is intentionally narrow, and why some features were marked as deferred instead of hidden behind partial abstractions. For WR2, this is a strength: the implementation remains aligned with the report, the frozen snapshot, and the current demonstration scope.

# Chapter 4

# Summary and Reflections

## 4.1 Team Members and the Division of Labor

The implementation-phase allocation used in WR2 preparation is summarised below. This allocation updates the earlier WR1 proposal roles and reflects the work packages needed to turn the concept into a demonstrable prototype and a traceable report pack.

- Guangjun HU: WR2 integration, requirements validation, appendix consolidation, and document traceability.
- Hongshuo HU: Unity architecture, scene wiring, core battle systems, and runtime integration.
- Cheng YE: UML and figure refinement, prototype presentation material, and supporting chapter drafting.
- Zehan CHEN: validation records, testing and meeting templates, and supporting documentation/website tasks.

## 4.2 Completed Process

By the end of this WR2 cycle, the project has moved from a proposal-level concept to a working prototype snapshot. The team produced a constrained architecture definition, implemented the first-pass gameplay modules, captured prototype screenshots, updated the runtime architecture to match the current code, and assembled a document pack that no longer depends on the incorrect WR1 skeleton originally stored in the Moodle WR2 folder. During closeout, the team also corrected a concrete prototype-accuracy gap by adding battle-end termination and textual outcome feedback, then backed that fix with a small EditMode test suite around the turn manager.

This reporting cycle also exposed an important process issue: several implementation records were not yet archived in a reusable structure. To correct that, WR2 now introduces a small project-record foundation covering meeting notes, process evidence notes, decision logs, validation notes, and testing/bug records. The cleaned GitHub repository now also shows the current closeout

workflow directly through merged document and code pull requests. These records are intentionally lightweight so that the team can continue using them in WR3 and the final report instead of rebuilding evidence from memory later.

## 4.3 Future Plan

The next stage connects directly to WR3 and the final report:

- strengthen battle-end presentation beyond the current top-bar/log feedback and tighten enemy-turn emphasis;
- improve onboarding cues for first-time interaction with the board and card selection flow;
- supplement the now-active GitHub PR history with any stronger task-board captures or confirmed attendance evidence the team can still recover;
- convert the current validation notes into a fuller WR3 testing pack with test cases, bug tracking, and retest records;
- expand the current automated coverage beyond battle-end behaviour into stable rule-heavy systems such as grid range checks and card resolution rules.

In short, WR2 has produced a stable prototype foundation, but it has also clarified what must be tightened before the software can be presented as a fully validated coursework submission.

# Bibliography


[1] Group 2. Tactical cards demo architecture constraints, 2026. Internal implementation note defining the WR2 prototype scope and module boundaries.

[2] Group 2. Tactical cards demo first pass scripts, 2026. Internal note translating the architecture constraints into the first implementation script set.

[3] Group 2. Tactical cards one-pager, 2026. Internal concept note describing the initial game direction, audience, and success criteria.

[4] University of Nottingham. Comp4135 coursework 1 specification, 2026. Local course specification used for WR2 scope, deadlines, and weekly report expectations.

[5] University of Nottingham. Comp4135 seminar 5: Prototyping, 2026. Seminar material on prototype goals, fidelity, and requirement validation.

[6] University of Nottingham. Comp4135 seminar 6: Software validation, 2026. Seminar material on validation, implementation design, and testing preparation.

[7] University of Nottingham. Comp4135 weekly report 2 template, 2026. Moodle WR2 template PDF used to recover the correct Weekly Report 2 chapter structure.

# Appendix A

# Meeting Minutes and Working Records

## A.1 Recovered Checkpoint (15/03/2026)

No formal implementation meeting minutes were archived in the local workspace for the first prototype checkpoint. To avoid inventing details, the following recovered checkpoint summary was compiled from dated artifacts that are present in the repository: the architecture constraints note, the first-pass script plan, and the prototype screenshots created on 15/03/2026.

- **Record type:** recovered implementation checkpoint compiled from local artifacts rather than formal minutes.
- **Date:** 15/03/2026.
- **Attendance:** exact attendance was not archived in the local workspace and should be replaced with confirmed team attendance if chat or calendar records are available.
- **Sources used:** `demo_architecture_constraints.md`, `demo_first_pass_scripts.md`, and prototype screenshots timestamped on 15/03/2026.
- **Main focus:** reduce the original game concept to one battle scene, four starter cards, a readable control loop, and a minimum set of scripts that can compile and run inside the existing Unity project.
- **Confirmed decisions:** single-battle scope, shared unit controller, deterministic enemy response, explicit card types, button-driven card interaction, and no campaign or deckbuilding systems in WR2.
- **Follow-up actions:** finish the first gameplay skeleton, refresh UML diagrams to match implementation, and gather prototype evidence for WR2.

## A.2 WR2 Evidence Freeze Note

The WR2 documentation pack uses an evidence freeze line of **02/04/2026 12:00**. This freeze is recorded in `wr2-snapshot-freeze-2026-04-02.md`. After that point, only changes that materially improve submission quality should be described in WR2. New features developed later should be documented in WR3 or the final report instead of being backfilled into this report.

## A.3 Supporting Record Index

The local record structure created during WR2 is summarised below and stored under `doc/src/project-records`.

- `wr2-asset-register.md`: reusable evidence list, missing items, suggested owners, and due times.
- `process/canonical-repo-evidence-2026-04-02.md`: canonical repository evidence note, discovery summary, and import checklist.
- `process/task-board-evidence-2026-04-02.md`: task-board evidence note, discovery summary, and import checklist.
- `process/discovery/README.md`: command-level discovery artifacts explaining what was checked from this workspace.
- `meetings/meeting-minutes-template.md`: lightweight meeting template for future weekly records.
- `meetings/implementation-checkpoint-recovered-2026-03-15.md`: transparent recovered note based on local artifacts.
- `decisions/wr2-implementation-decisions.md`: decision log extracted from the current implementation direction.
- `validation/wr2-heuristic-walkthrough-2026-04-01.md`: internal WR2 walkthrough and findings.
- `testing/wr3-prep-log-2026-04-01.md`: initial test and bug tracking preparation note for the next report cycle.

These records are intentionally lightweight. Their purpose is to stop evidence loss between WR2 and WR3, not to pretend that the record-keeping process was already complete earlier in the prototype cycle.